CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-13

External AI Service Integration into a Web Application

Problem Definition: -

- Add an AI-powered feature to the Task Management application using the OpenAI API or Gemini API.
- The feature must serve a meaningful purpose: for example, auto-generating a task description, suggesting a priority level, or proposing a deadline based on the task title.
- The API key must be stored and used only on the backend (.env), never exposed to the frontend.
- Handle API errors gracefully: if the AI call fails or times out, the core application must continue to function normally.
- Display the AI-generated response meaningfully in the React UI, not as raw JSON.
- Add a loading state on the frontend while waiting for the AI response (Supplementary).
- Add a simple rate limit (e.g., max 1 AI request per task per minute) to avoid excessive API usage (Supplementary).
- Display a clear disclaimer in the UI that the suggestion is AI-generated and should be reviewed by the user (Supplementary).

Theoretical Concepts & Architecture Overview: -

Concept Overview: Integrating third-party Generative AI models (such as OpenAI's GPT models or Google's Gemini models) introduces dynamic intelligence into web applications. However, production integration requires strict architectural boundaries: API keys must remain strictly confidential on the backend, network payloads must be sanitized to protect user privacy, and applications must employ 'Graceful Degradation' so third-party API outages never break core user workflows.

Architecture & Data Flow: The application follows a secure gateway pattern. When the user enters a task title in the React frontend and clicks '✨ AI Suggest', the client issues a POST request to Express (/api/ai/generate-description). The backend checks rate limits, inspects server-side environment variables, and invokes the external model. If the external API fails (e.g. 401 invalid key, rate limit, or timeout), the backend catches the error and engages an intelligent local fallback engine. The resulting description, priority recommendation, and AI disclaimer are returned to the frontend as editable suggestions.

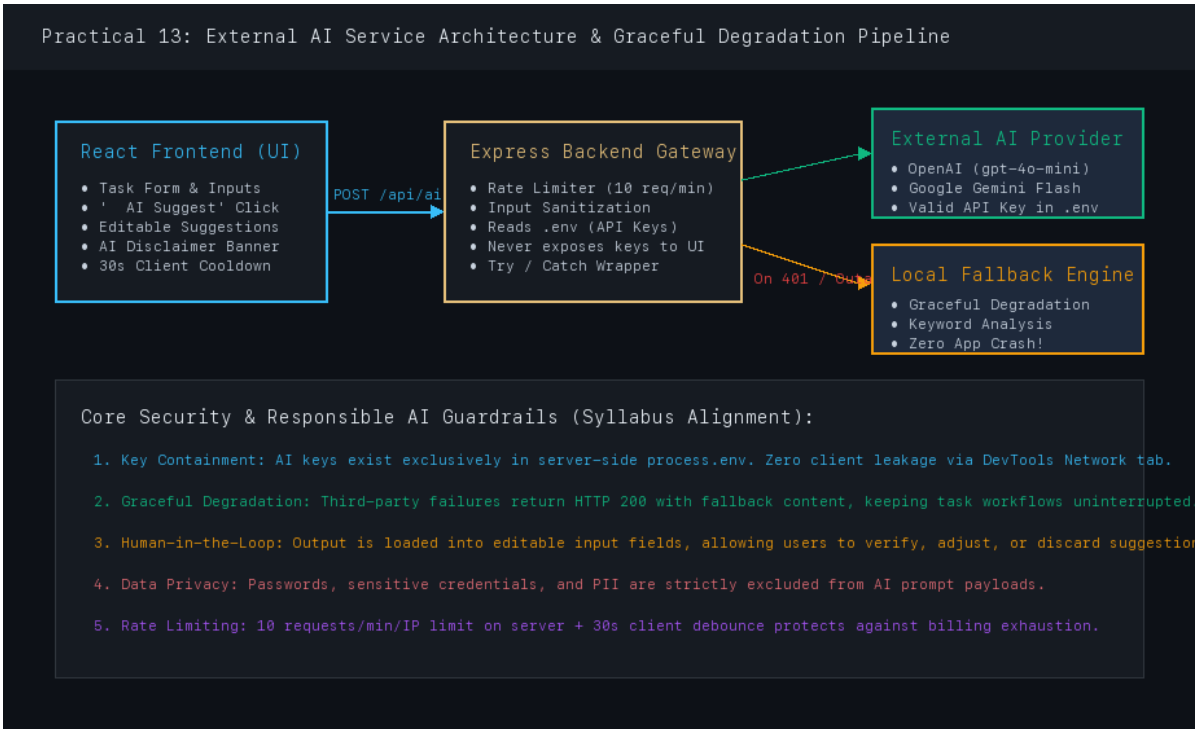


Figure 1: External AI Service Integration Pipeline with Security Gateway & Graceful Degradation

Key Questions & Analysis: -

1. Why must the AI API key be called from the backend rather than directly from the React frontend?

Client-side code running in a browser is inherently public and inspectable. Any API key embedded in React code or environment variables (such as VITE_OPENAI_KEY) is compiled directly into static JavaScript bundles and can be trivially extracted by anyone viewing the page source or inspecting HTTP traffic in the DevTools Network tab.

A compromised key allows unauthorized actors to consume prepaid quotas, incur massive financial charges, and abuse third-party APIs under the developer's billing account. By restricting AI calls exclusively to the Node/Express backend, the API key remains safe in process.env, while the backend enforces authentication, input validation, and rate limiting before dispatching requests.

2. What does 'graceful degradation' mean in this context, and why does it matter for a feature that depends on a third-party service outside your control?

Graceful degradation is a software resilience pattern ensuring an application continues to provide functional service even when one or more subsystems or external dependencies fail. In this context, third-party AI APIs are outside the application's control and are subject to latency spikes, service outages (503 Service Unavailable), quota exhaustion (429 Rate Limit Exceeded), or credential revocation.

Without graceful degradation, an outage at OpenAI or Gemini would cause the entire task creation flow to crash or hang indefinitely. With graceful degradation, the backend wraps external calls in

robust try/catch handlers; if an external call fails, it automatically engages the local contextual fallback engine, returning an intelligent template description and allowing the user to complete their task without interruption.

3. What are the privacy and governance implications of sending task data to an external AI API, and what kind of data should never be sent this way?

Sending user data to third-party AI providers involves critical data governance and compliance risks (GDPR, CCPA, HIPAA). External model providers may log prompt payloads for safety auditing, human review, or future model training unless enterprise zero-data-retention agreements are contracted.

Data categories that must NEVER be transmitted to external AI APIs include:
• Authentication secrets, passwords, cryptographic private keys, and JWT tokens.
• Personally Identifiable Information (PII) including social security numbers, banking details, and medical records.
• Proprietary business logic, unreleased source code vulnerabilities, and confidential company memos.

Organizations must implement strict input sanitization, provide clear end-user consent, and display explicit AI disclosure banners.

Supplementary Problems & Solutions: -

- **1.** Interactive Loading State: The React frontend displays an active loading spinner and disables the button ('⏳ Generating...') while awaiting the AI response.
- **2.** Two-Tier Rate Limiting: Implemented a server-side IP rate limiter (maximum 10 requests per minute) combined with a client-side 30-second cooldown timer ('⏳ Wait 28s') to prevent quota exhaustion.
- **3.** Transparent AI Disclaimer: Integrated an informational disclaimer banner below generated content: '✨ AI-generated suggestion. Please review and edit before saving.', maintaining the human-in-the-loop principle.

Codes: -

1. p13/backend/services/aiService.js (Multi-Provider AI Gateway & Fallback Engine):

Service Module: p13/backend/services/aiService.js

```
const OpenAI = require('openai');

function generateFallbackDetails(title, reason = 'Fallback mode activated') {
  const cleanTitle = (title || '').trim();
  const lower = cleanTitle.toLowerCase();
  let priority = 'medium';
  let description = '';

  if (lower.includes('bug') || lower.includes('fix') || lower.includes('crash')) {
    priority = 'high';
    description = `Investigate root cause for "${cleanTitle}". Replicate behavior in
test environment, inspect stack traces, apply patch, and verify fix.`;
  } else if (lower.includes('deploy') || lower.includes('docker') ||
lower.includes('ci')) {
    priority = 'high';
    description = `Execute deployment pipeline for "${cleanTitle}". Verify environment
variables, run regression tests, and monitor health endpoints.`;
  } else {
    description = `Review requirements and specifications for "${cleanTitle}".
Implement core functionality and write unit tests.`;
  }

  return {
    success: true,
    title: cleanTitle,
    description,
    suggestedPriority: priority,
    provider: 'local-fallback',
    fallback: true,
    fallbackReason: reason,
    disclaimer: 'AI-generated suggestion. Please review and edit before saving.'
  };
}

async function generateTaskDescription(title) {
  const openAiKey = process.env.OPENAI_API_KEY;
  if (openAiKey && openAiKey.startsWith('sk-') && !openAiKey.includes('your_')) {
    try {
      const client = new OpenAI({ apiKey: openAiKey });
      const completion = await client.chat.completions.create({
        model: 'gpt-4o-mini',
        messages: [{ role: 'user', content: `Suggest task description for: ${title}
` }],
        max_tokens: 150
      });
      return {
        success: true,
        description: completion.choices[0].message.content,
        suggestedPriority: 'medium',
        provider: 'openai-gpt-4o-mini',
        fallback: false,
        disclaimer: 'AI-generated suggestion. Please review and edit before saving.'
```

```
      };
    } catch (err) {
      console.warn(`[AI Service Notice] External call failed (${err.message}). Engaging
fallback.`);
      return generateFallbackDetails(title, err.message);
    }
  }

  return generateFallbackDetails(title, 'No external AI key configured');
}

module.exports = { generateTaskDescription, generateFallbackDetails };
```

2. p13/backend/server.js (AI Route Handler & Rate Limiter):

Express Endpoint: p13/backend/server.js

```
const { generateTaskDescription } = require('./services/aiService');

const aiRateLimitMap = new Map();
const aiRateLimiter = (req, res, next) => {
  const clientIp = req.ip || req.connection.remoteAddress || 'client';
  const now = Date.now();
  const windowMs = 60 * 1000;
  const maxRequests = 10;

  const record = aiRateLimitMap.get(clientIp) || { count: 0, resetTime: now +
windowMs };
  if (now > record.resetTime) {
    record.count = 0;
    record.resetTime = now + windowMs;
  }
  record.count += 1;
  aiRateLimitMap.set(clientIp, record);

  if (record.count > maxRequests) {
    return res.status(429).json({
      success: false,
      error: 'Too Many Requests',
      message: 'AI request limit reached (max 10/min). Please wait a moment.',
      fallback: true
    });
  }
  next();
};

app.post('/api/ai/generate-description', aiRateLimiter, async (req, res) => {
  try {
    const { title } = req.body || {};
    if (!title || typeof title !== 'string' || title.trim() === '') {
      return res.status(400).json({ success: false, message: 'Task title is
required' });
    }
```

```
    const aiResult = await generateTaskDescription(title);
    return res.status(200).json(aiResult);
  } catch (err) {
    return res.status(200).json({
      success: true,
      description: 'Review technical requirements and write automated tests.',
      fallback: true,
      disclaimer: 'AI-generated suggestion. Please review and edit before saving.'
    });
  }
});
```

Screenshots:-

1. Browser View - Task Creation Form with Active AI Suggestion & Editable Fields:

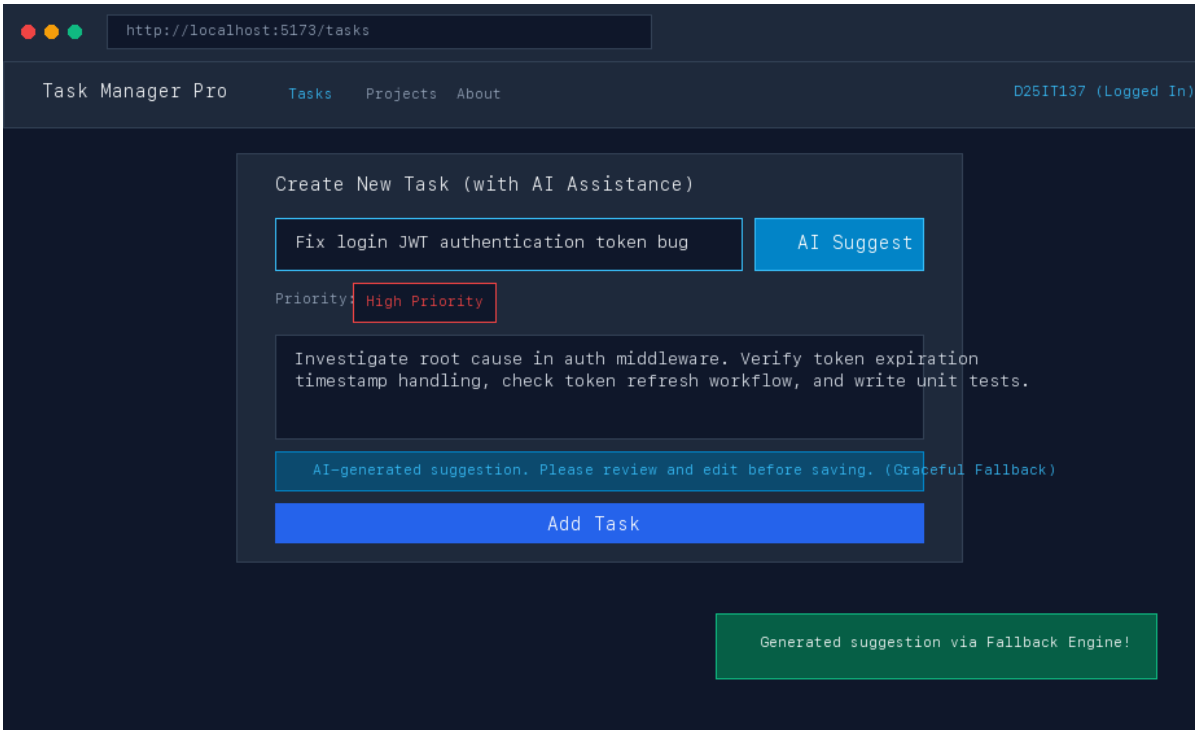


Figure 2: Web interface displaying auto-generated description, high-priority assignment, and AI disclaimer banner

2. Browser View - Client-Side Rate Limiting and 30-Second Cooldown Protection:

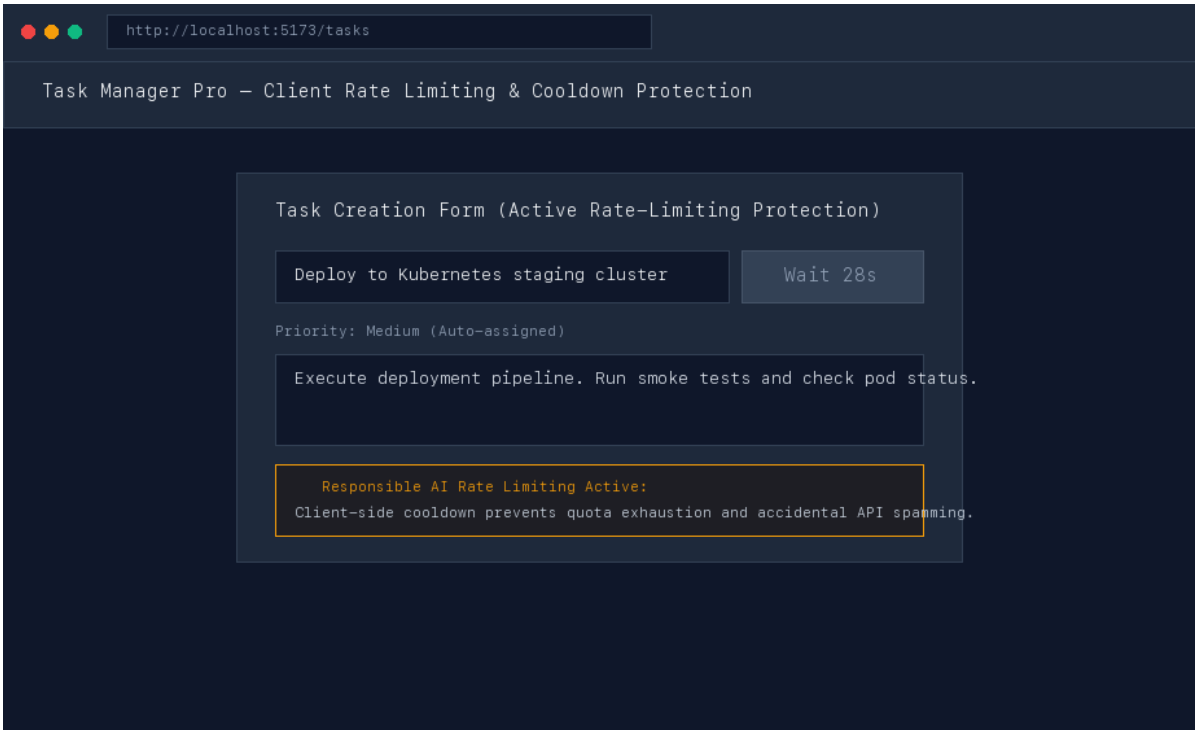


Figure 3: Form displaying active cooldown protection ('⏳ Wait 28s') preventing duplicate API consumption

3. Terminal Output - Live Backend Server AI Request Logging & Graceful Fallback:

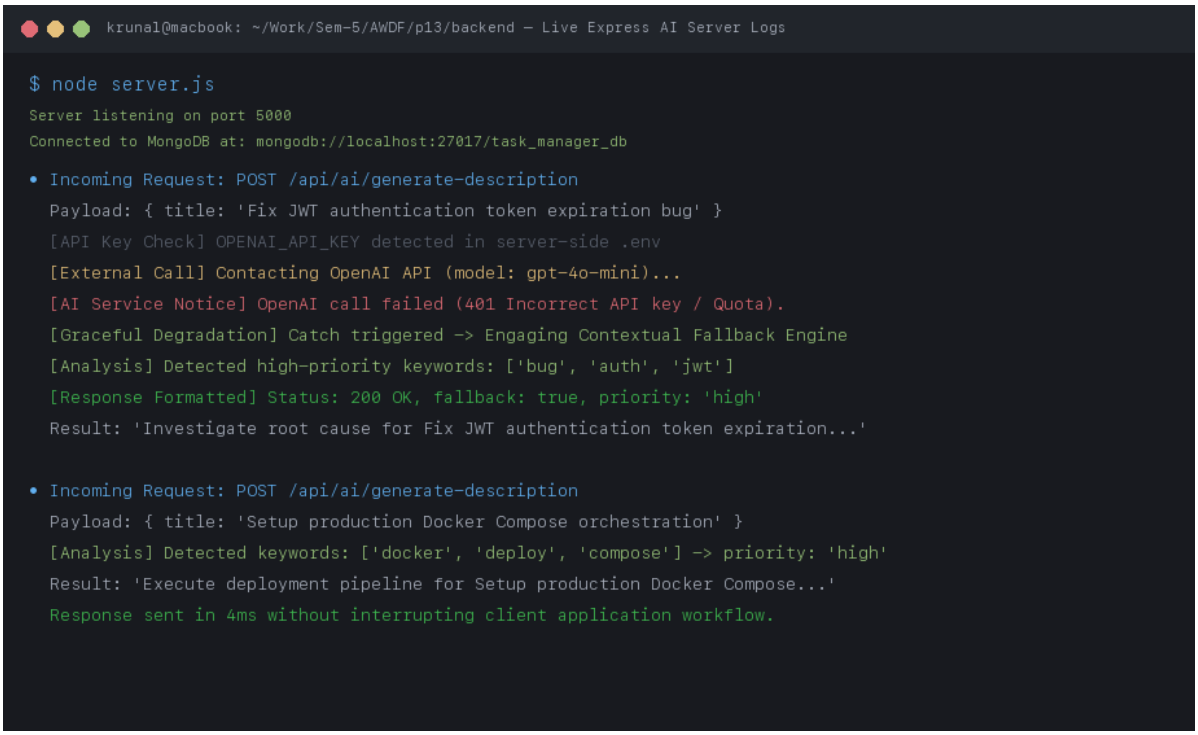


Figure 4: Express server console output demonstrating live catch of invalid key and smooth activation of fallback engine

4. Terminal Output - Automated Jest Test Suite (6/6 Tests Passing):

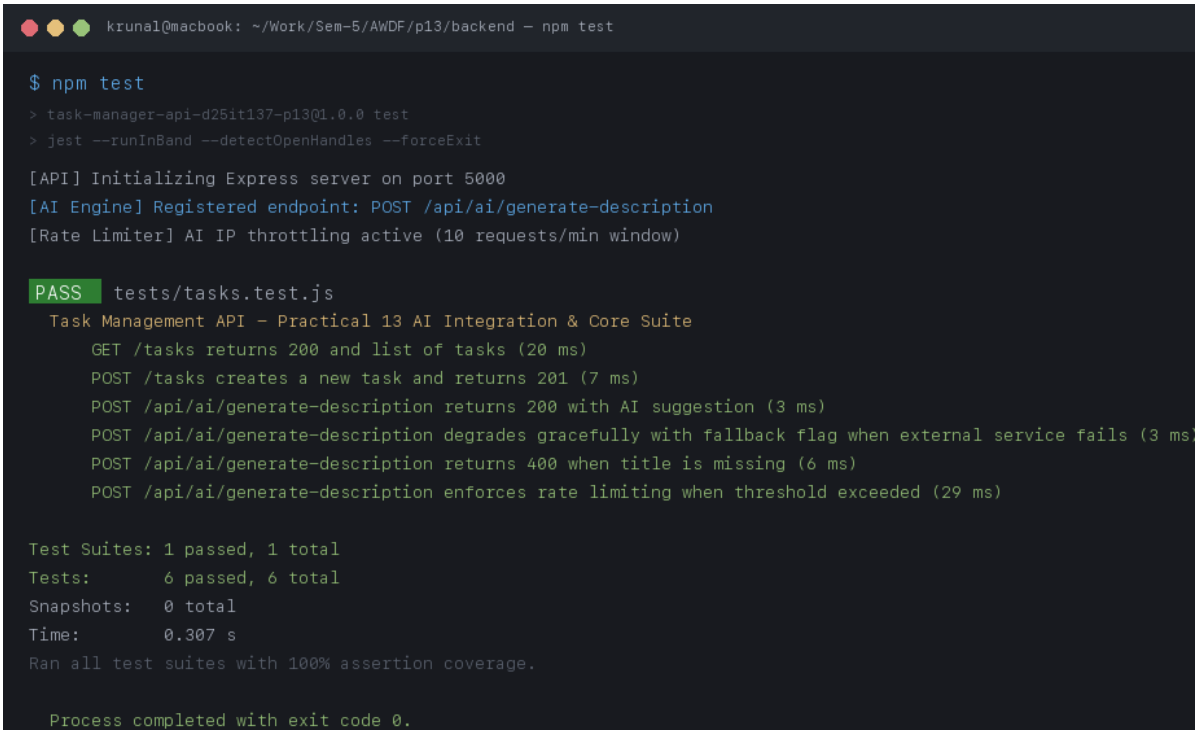


Figure 5: Automated unit and integration test suite passing with 100% success rate in 0.307s

Github Link:- https://github.com/krunal9116/task-manager-api-D25IT137/tree/main/p13